
S4Utils

Release 0.0.1

Mathieu Jeannin, Paul Goulain

Jul 22, 2020

CONTENTS:

1	What is it?	1
2	Documentation Contents	3
2.1	Download and installation	3
2.2	Tutorials	6
3	API Reference	23
3.1	Material database	23
3.2	S4Utils functions references	26
3.3	epsInterpolator functions references	31
4	Indices and tables	33
	Python Module Index	35
	Index	37

WHAT IS IT?

S4Utils is a collection of python utility functions to ease the use of the [S4](#) software¹

References

¹ Liu, V. and Fan, S., *Computer Physics Communications* **183**, 2233-2244 (2012) [10.1016/j.cpc.2012.04.026](#)

DOCUMENTATION CONTENTS

2.1 Download and installation

Here are the guidelines to install S⁴ and the Python API either as a *Conda package* or *Building from source*

2.1.1 Installing S⁴ and the Python API

Conda package

If you run Anaconda, S⁴ can be installed as a conda package. From an anaconda prompt

```
conda config --add channels conda-forge
conda upgrade conda
```

To add the channel conda-forge as one of the default channel for downloading packages and upgrade all you packages. Create a new dedicated conda environment (*e.g* with python 3.7:

```
conda create -n s4 python=3.7
```

Activate the environment and install the package:

```
conda activate s4
conda install -c paulgoulain s4
```

You should be able to import the S⁴ module in a python prompt:

```
import S4 as S4
```

Building from source

You can download S⁴ on github, however we recommend using slightly more up-to-date forks of the original code, by cloning *e.g.*

```
git clone https://github.com/phoebe-p/S4
cd S4
```

or

```
git clone https://github.com/marcus-o/S4
cd S4
```

Windows

To be added

Linux

Two options are proposed, depending on which optional libraries you use for the linear algebra (BLAS and Lapack), Fourier transform (FFT) and matrix factorization (CHOLMOD).

The first one makes use of the (free but proprietary) [Math Kernel Library \(MKL\)](#) from [Intel](#), which largely improves performances (at least on Intel CPUs). [jump](#)

The second one uses only open-source libraries ([OpenBlas](#), [FFTW3](#) and [CHOLMOD](#) ([SuiteSparse](#))), at the risk of lower performances. [jump](#)

Proprietary option

The first step is to install the [MKL](#) libraries:

```
sudo bash
# <type your user password when prompted. this will put you in a root shell>
# cd to /tmp where this shell has write permission
cd /tmp
# now get the key:
wget https://apt.repos.intel.com/intel-gpg-keys/GPG-PUB-KEY-INTEL-SW-PRODUCTS-2019.PUB
# now install that key
apt-key add GPG-PUB-KEY-INTEL-SW-PRODUCTS-2019.PUB
# now remove the public key file exit the root shell
rm GPG-PUB-KEY-INTEL-SW-PRODUCTS-2019.PUB
exit
# add the repository
sudo sh -c 'echo deb https://apt.repos.intel.com/mkl all main > /etc/apt/sources.list.
↪d/intel-mkl.list'
# update the package list
sudo apt update
```

You can now install one of the MKL releases. Since you have to specify the component you wish to install, you can [check for the version list](#)

```
## exemple intel-mkl-2020.0-088
sudo apt install intel-mkl-<VERSION>.<UPDATE>-<BUILD_NUMBER>
```

We also need the *OpenMP* and *suitesparse-dev* libraries:

```
# OpenMP
sudo apt install libomp-dev
# CHOLMOD
sudo apt-get install libsuitesparse-dev
```

The correct compilation flags have to be set in the *Makefile* to use the MKL libraries. Edit the following lines if necessary, the other options should work:

```
BLAS_LIB = -lmkl_rt
LAPACK_LIB = -lmkl_rt

FFTW3_INC =
FFTW3_LIB = -lfftw3
```


Then from a command prompt in the *S4* folder:

```
make boost
make S4_pyext
```

Note: You might have to adapt the end of the Makefile to use the correct `pip` installer. `pip3` is used by default, but if you want to install *S4* in a particular environment, you should first activate it before running `make S4_pyext`.

In case an error occurs, it is most probably because the *MKL* libraries were installed somewhere where the compiler does not find them (*e.g* in `/opt/`). Either use the provided `.sh` script in the *bin* subdirectory of the installation directory (following [this link](#)):

```
$ (MKLdir)/bin/compilervars.sh
```

Or manually link them using (might require *sudo*):

```
ln -s $(MKLdir)/*.so /usr/lib/$(where_to_store)
```

i.e in my case:

```
sudo ln -s /opt/intel/compilers_and_libraries/linux/mkl/lib/intel64/*.so /usr/lib/x86_
↪64-linux-gnu/
```

Everything should compile without any errors or warnings.

Open source option

We start by installing the OpenBlas and CHOLMOD libraries:

```
## linear algebra blas and lapack
sudo apt-get install libopenblas-dev
## fourier transform fftw3
sudo apt-get install libfftw3-dev
## CHOLMOD for some fourier decompositions
sudo apt-get install libsuitesparse-dev
```

Check that the Makefile contains the following lines (all other options should already be set):

```
BLAS_LIB = -lblas
LAPACK_LIB = -llapack

FFTW3_INC =
FFTW3_LIB = -lfftw3
```

Then compile:

```
make boost
make S4_pyext
```

Note: You might have to adapt the end of the Makefile to use the correct `pip` installer. `pip3` is used by default, but if you want to install *S4* in a particular environment, you should first activate it before running `make S4_pyext`.

Everything should compile without any errors or warnings.

2.1.2 Installing S4Utils

Installing S4Utils only requires `pip`. From a command prompt in the main S4Utils directory:

```
pip install ./
```

Should run without any trouble. The last tutorial script *Tutorial 4: Dispersive Metal-Insulator-Metal grating* (download script) uses some of S4Utils functions and should thus run if the installation is successful.

2.2 Tutorials

2.2.1 Tutorial 1: A single, flat interface

In this example, we compute the reflection and transmission coefficients at a single, flat interface between a superstrate (incident medium) and a substrate. The example file can be found under [examples/FresnelCoefficients.py](#). The simulation can be run either in two or three dimensions, to demonstrate the syntax adjustments in creating the S⁴ simulation object.

Once S4 has been installed, you should be able to import it, along with `numpy` and `matplotlib.pyplot`:

```
import S4 as S4
import numpy as np
import matplotlib.pyplot as plt
```

As stated in the [S4 documentation](#), “S⁴ solves the linear Maxwell’s equations, which are scale-invariant. Therefore, S⁴ uses normalized units, so the conversion between the numbers in S4 and physically relevant numbers can sometimes be confusing.” In order to clearly link the reduced units and the actual physical ones, we keep track of the speed of light (assumed to be 1 in S⁴):

```
c_const = 3e8
```

and the general scale is defined by the period of the lattice:

```
a = 1 ## period, normalized units. This defines the length scale
```

So if we (arbitrarily) decide that a here is in units of micrometers, we can run the simulation at a wavelength of 1 micron by setting the following:

```
lbda = 1e-6 ## say we compute at 1µm wavelength
f = c_const/lbda ## frequency in SI units
f0 = f/c_const*1e-6 ## so the reduced frequency is f/c_const*a[SI units]
```

We pick the dimension of the simulation (2 or 3), then we set the lattice period and the number in-plane Fourier expansions order to be used (`S4.New`):

```
Dimension = 3 # 2 or 3

if Dimension == 2:
    ### 2 dimensional simulation, 1D lattice
    S = S4.New(Lattice = a,
               NumBasis = 1)
elif Dimension == 3:
    ### 3 dimensional simulation, 2D lattice
    S = S4.New(Lattice = ((a,0), (0,a)),
               NumBasis = 1)
```

In the case of a 2D simulation, the lattice is one-dimensional and has a simple period a . For the 3D simulation, the lattice dimension is two and we choose a square lattice of period a in both dimensions. Since the layer is unpatterned, we only need to compute the 0th plane wave order, and hence NumBasis=1 is enough.

We then setup the main parameters of the simulation: the angle of incidence θ , and the refractive indices of the superstrate and substrate:

```
theta = np.arange(0,89,2) ## angle of incidence in degree

n_superstrate = 1 ## incident medium index
eps_superstrate = n_superstrate**2 ## incident medium permittivity
n_substrate = 2 ## substrate index
eps_substrate = n_substrate**2 ## substrate permittivity
```

S⁴ uses (possibly complex and anisotropic) permittivities instead of refractive index, which is used when declaring the materials used in the simulation (S.SetMaterial):

```
S.SetMaterial(Name='Air', Epsilon = eps_superstrate)
S.SetMaterial(Name='Substrate', Epsilon = eps_substrate)
```

The names used in declaring the materials ('Air' and 'Substrate') will be used to set the permittivities of the layers that we declare, setting their names and thickness (S.AddLayer):

```
AirThick = 1 ## in reduced units, same scale as a
SubThick = 1 ## in reduced units, same scale as a

S.AddLayer(Name='Air', Thickness=AirThick, Material='Air')
S.AddLayer(Name='Substrate', Thickness=SubThick, Material='Substrate')
```

The layers are added in order, from the top (incident layer) to the bottom (substrate) one.

At this point the simulation is set up, and we can now run it to retrieve the reflection and transmission coefficients. We initialize the arrays storing the results:

```
Rte = np.zeros(len(theta))
Rtm = np.zeros(len(theta))
Tte = np.zeros(len(theta))
Ttm = np.zeros(len(theta))
```

and set the (single) frequency at which the simulation runs (S.SetFrequency):

```
S.SetFrequency(f0)
```

The bulk of the calculation is in the main loop over the angle of incidence:

```
for ii, thi in enumerate(theta):
    #####
    ## TM excitation
    S.SetExcitationPlanewave(
        IncidenceAngles=(thi,0), #S4 names are reversed (phi,theta)
        sAmplitude=0.,
        pAmplitude=1.,
        Order=0)
    inc, r = S.GetPowerFlux(Layer='Air', zOffset=0)
    fw, _ = S.GetPowerFlux(Layer='Substrate', zOffset=SubThick)
    Rtm[ii] = np.abs(-r/inc)
    Ttm[ii] = np.abs(fw/inc)
```

(continues on next page)

(continued from previous page)

```
#####
## TE excitation
S.SetExcitationPlanewave(
    IncidenceAngles=(thi,0), #S4 names are reversed (phi,theta)
    sAmplitude=1.,
    pAmplitude=0.,
    Order=0)
inc, r = S.GetPowerFlux(Layer='Air', zOffset=0)
fw, _ = S.GetPowerFlux(Layer='Substrate', zOffset=SubThick)
Rte[ii] = np.abs(-r/inc)
Tte[ii] = np.abs(fw/inc)
```

The exciting wave properties are set through the `S4.SetExcitationPlanewave` method. The arguments are the couple of incidence angles (θ, ϕ) in spherical coordinates which represent the polar angle and the azimuthal angle. (Note that the S4 documentation uses the opposite denomination), the amplitude of the s - and p -components of the electric field, and the order (defaults to 0, see doc). Note that here we use the `enumerate` function from python which allows us to pass directly the local variable `thi` to the plane wave function.

We use the method `S.GetPowerFlux` which returns the integral of the z component of the Poynting vector over unit cell surface normal to the z direction, decomposed into forward and backward propagating modes. Hence, the incident and reflected power is obtained using:

```
inc, r = S.GetPowerFlux(Layer='Air', zOffset=0)
```

so the calculation is lead in the first layer. The `zOffset` parameter specifies the vertical offset (in reduced units) from the *front* surface of the layer. This only matters for lossy layers. Conversely, the transmitted power is obtained with:

```
fw, _ = S.GetPowerFlux(Layer='Substrate', zOffset=SubThick)
```

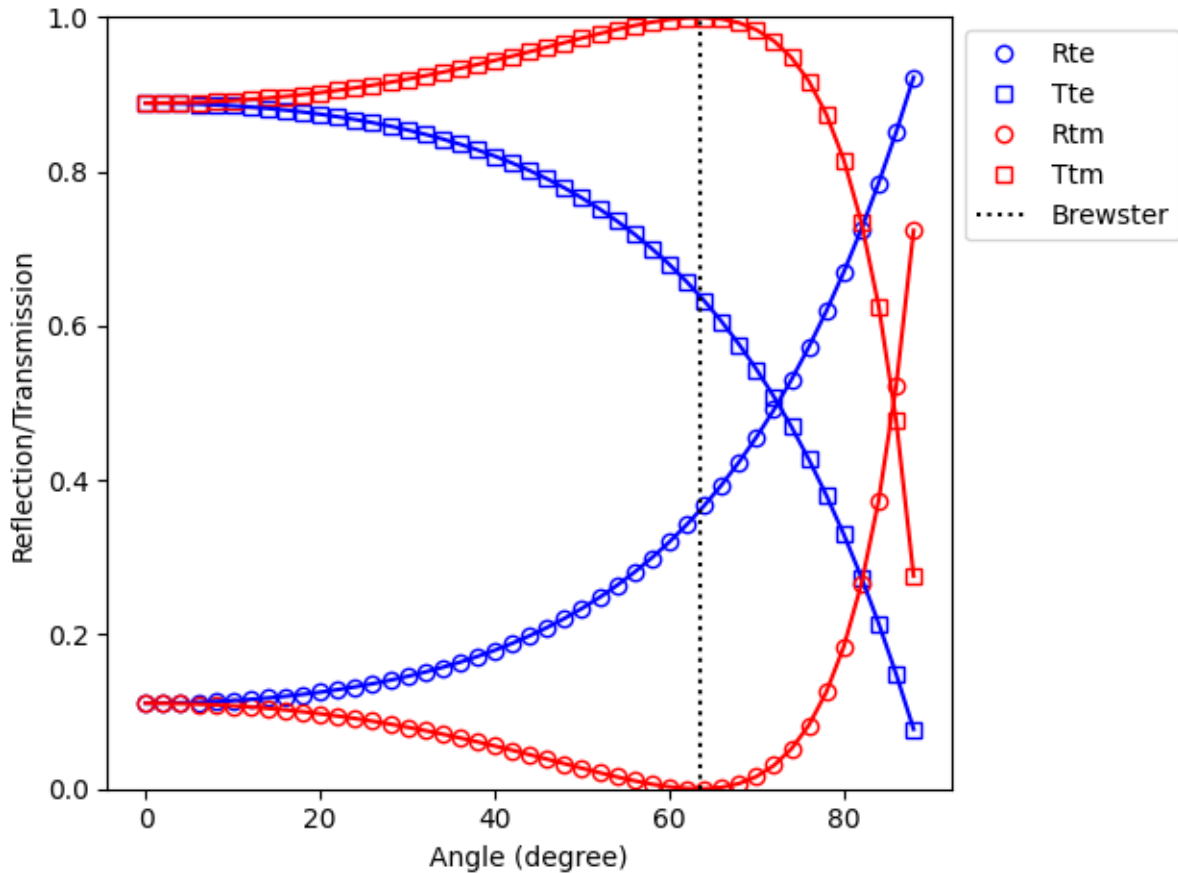
where we discard the value of the backward propagating wave from the substrate, since it should be 0 by definition. Note that we specified a `zOffset` equal to the layer thickness only for the sake of clarity.

As stated in the [S4 documentation](#), the incident power is only unity at normal incidence. Hence, the values extracted from the simulation have to be normalized by the input power:

```
Rtm[ii] = np.abs(-r/inc)
Ttm[ii] = np.abs(fw/inc)
```

(note the sign convention for the reflected power) and we take the absolute value `abs` to cast the (otherwise complex) value returned by the simulation to a real number.

We can plot the results and compare to the Fresnel coefficients formula, as well as the Brewster angle:



where the symbols are the results of the simulations, and the solid lines the analytical results.

2.2.2 Tutorial 2: A single slab, or Fabry-Pérot etalon

We now quickly move on to a similar, simple example demonstrating the calculation of the transmission and reflection from a single dielectric slab, either as a function of angle of incidence or as a function of frequency. The example file is under [examples/FabryPerot.py](#). The beginning of the script is exactly the same, setting the base unit and the dimensionality of the simulation:

```
a = 1 ## period, normalized units. This defines the length scale
Dimension = 3 # or 3
if Dimension == 2:
    ### 2 dimensional simulation, 1D lattice
    S = S4.New(Lattice = a,
               NumBasis = 1)
elif Dimension == 3:
    ### 3 dimensional simulation, 2D lattice
    S = S4.New(Lattice = ((a,0), (0,a)),
               NumBasis = 1)
```

and then the permittivities, material objects, thickness and layer objects:

```
n_superstrate = 1 ## incident medium index
eps_superstrate = n_superstrate**2 ## incident medium permittivity
```

(continues on next page)

(continued from previous page)

```
n_slab = 2 ## substrate index
eps_slab = n_slab**2 ## substrate permittivity

S.SetMaterial(Name='Air', Epsilon = eps_superstrate)
S.SetMaterial(Name='Slab', Epsilon=eps_slab)

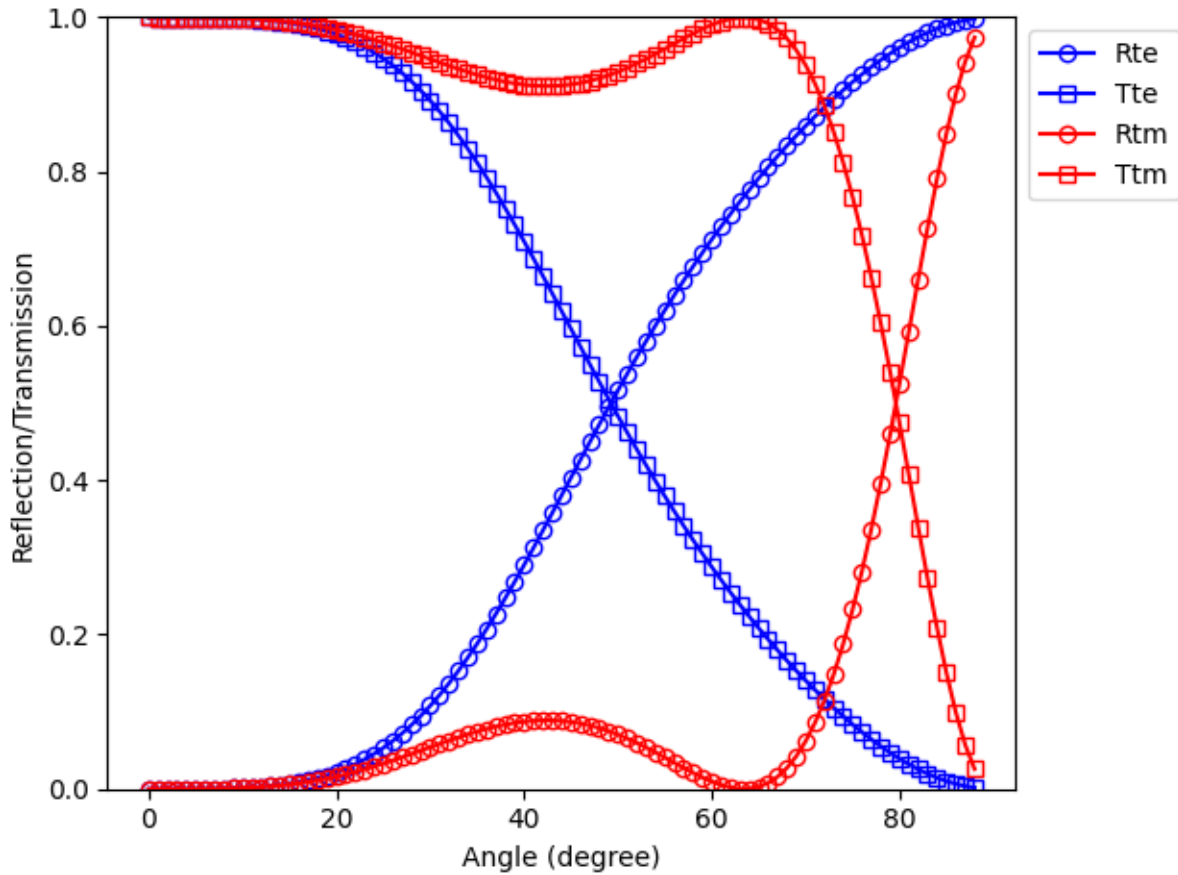
AirThick = 1
SlabThick = 1

S.AddLayer(Name='AirTop', Thickness=AirThick, Material='Air')
S.AddLayer(Name='Slab', Thickness=SlabThick, Material='Slab')
S.AddLayer(Name='AirBottom', Thickness=AirThick, Material='Air')
```

Now two sets of calculations are possible. In the first one, we sweep over the angle of incidence as in the previous example. To do so, we first define a frequency at which we wish to run the calculation. Again, assuming that a above is in micrometers, we can define *e.g.* a 1 micron wavelength and the corresponding frequencies, both in SI units (f) and reduced units($f0$) and the angular range:

```
lbda = 1e-6 ## say we compute at 1µm wavelength
f = c_const/lbda ## frequency in SI units
f0 = f/c_const*1e-6 ## so the reduced frequency is f/c_const*a[SI units]
theta = np.arange(0,89,1)
S.SetFrequency(f0)
```

The rest is exactly the same as above, and we obtain the angular reflectivity and transmittivity:



In the second set of calculation, we fix the angle of incidence and set up a spectral range over which we run the calculation:

```
theta = 15 ## fixed angle of incidence

lbda = np.linspace(400e-9,1e-6,200) ## 400nm to 1μm
f = c_const/lbda ## ferquency in SI units
f0 = f/c_const*1e-6 ## so the reduced frequency is f/c_const*a[SI units]
```

While the bulk of the code is the same, note that now we must remember to include the `S.SetFrequency` method in the main loop:

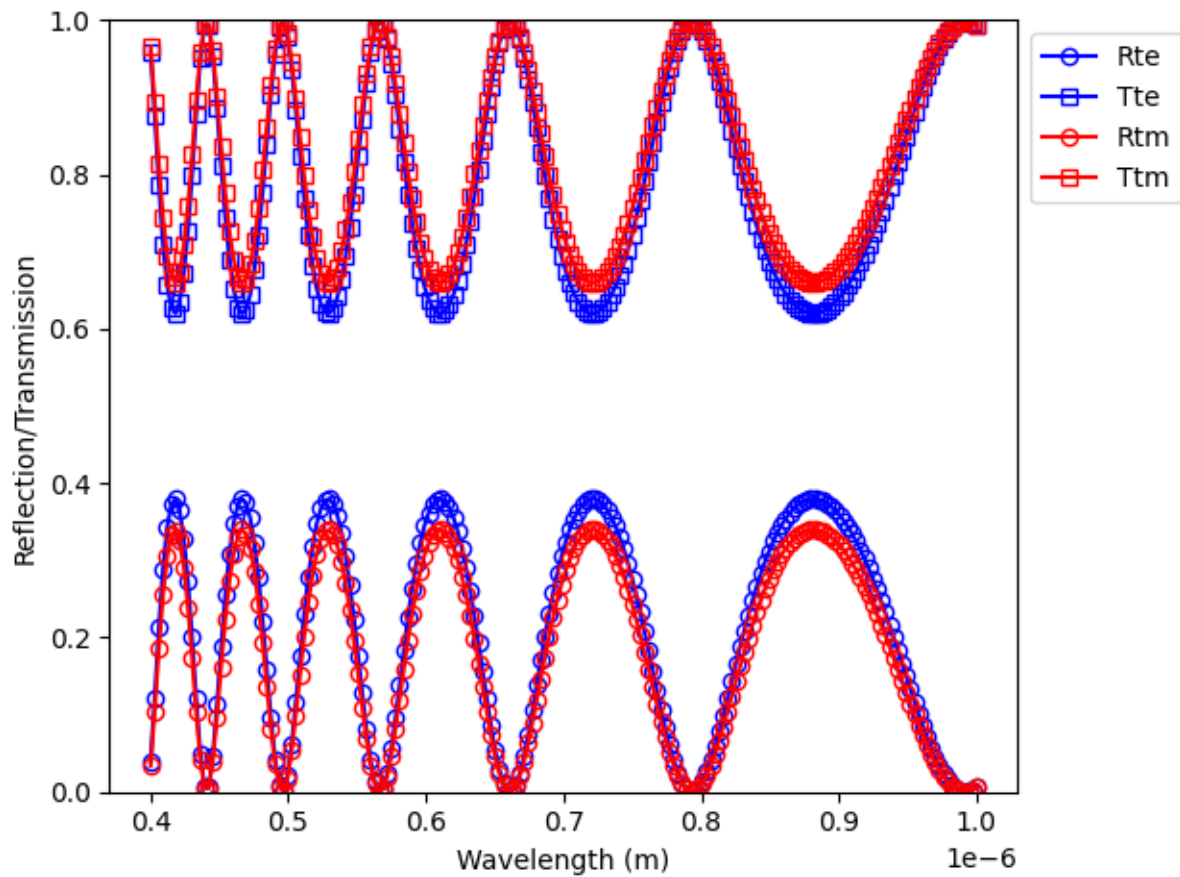
```
for ii, fi in enumerate(f0):
    S.SetFrequency(fi)
    #####
    ## TM excitation
    S.SetExcitationPlanewave(
        IncidenceAngles=(theta,0), #S4 names are reversed (phi,theta)
        sAmplitude=0.,
        pAmplitude=1.,
        Order=0)
    inc, r = S.GetPowerFlux(Layer='AirTop', zOffset=0)
    fw, _ = S.GetPowerFlux(Layer='AirBottom', zOffset=AirThick)
    Rtm[ii] = np.abs(-r/inc)
    Ttm[ii] = np.abs(fw/inc)
```

(continues on next page)

(continued from previous page)

```
#####
## TE excitation
S.SetExcitationPlanewave(
    IncidenceAngles=(theta,0), #S4 names are reversed (phi,theta)
    sAmplitude=1.,
    pAmplitude=0.,
    Order=0)
inc, r = S.GetPowerFlux(Layer='AirTop', zOffset=0)
fw, _ = S.GetPowerFlux(Layer='AirBottom', zOffset=AirThick)
Rte[ii] = np.abs(-r/inc)
Tte[ii] = np.abs(fw/inc)
```

which quickly allows to plot the results:



2.2.3 Tutorial 3: Calculating absorption

Before moving to a more complex simulation, let's apply the knowledge we have so far to extract a more subtle quantity from a simple simulation: the absorption of light inside a given layer. The example file is under [examples/Absorption.py](#).

We set up a very basic 2D simulation, without any pattern, containing a superstrate, an absorbing slab and either a transparent or reflecting substrate:

```
f = 1 ## single frequency in reduced units
a = 1 ## period
S = S4.New(Lattice=a, NumBasis=1) ## Simple lattice

SlabThick = 2 ## thickness of absorbing slab
AirThick = 1 ## 0 should work also...

n = 3 ## slab index, real part
k = 0.01 ## slab index, imaginary part
epsSlab = (n+1.0j*k)**2 ## slab permittivity
epsMirr = -1e10 ## mirror, to mimic a metal
```

where a very large, real negative permittivity can be used to mimic a lossless metal. We will perform the computation at a single frequency but for several angles:

```
theta = np.arange(0, 89, 2)
```

and the slab is either surrounded by air or backed by a mirror:

```
S.SetMaterial(Name='Air', Epsilon=(1.0+1.0j*0))
S.SetMaterial(Name='Slab', Epsilon=epsSlab)
# S.SetMaterial(Name='Mirror', Epsilon=epsMirr)

S.AddLayer(Name='AirTop', Thickness=AirThick, Material='Air')
S.AddLayer(Name='Slab', Thickness=SlabThick, Material='Slab')
# either mirror or air for substrate
# S.AddLayer(Name='AirBottom', Thickness=AirThick, Material='Mirror')
S.AddLayer(Name='AirBottom', Thickness=AirThick, Material='Air')
```

The main loop looks similar to the previous simulations:

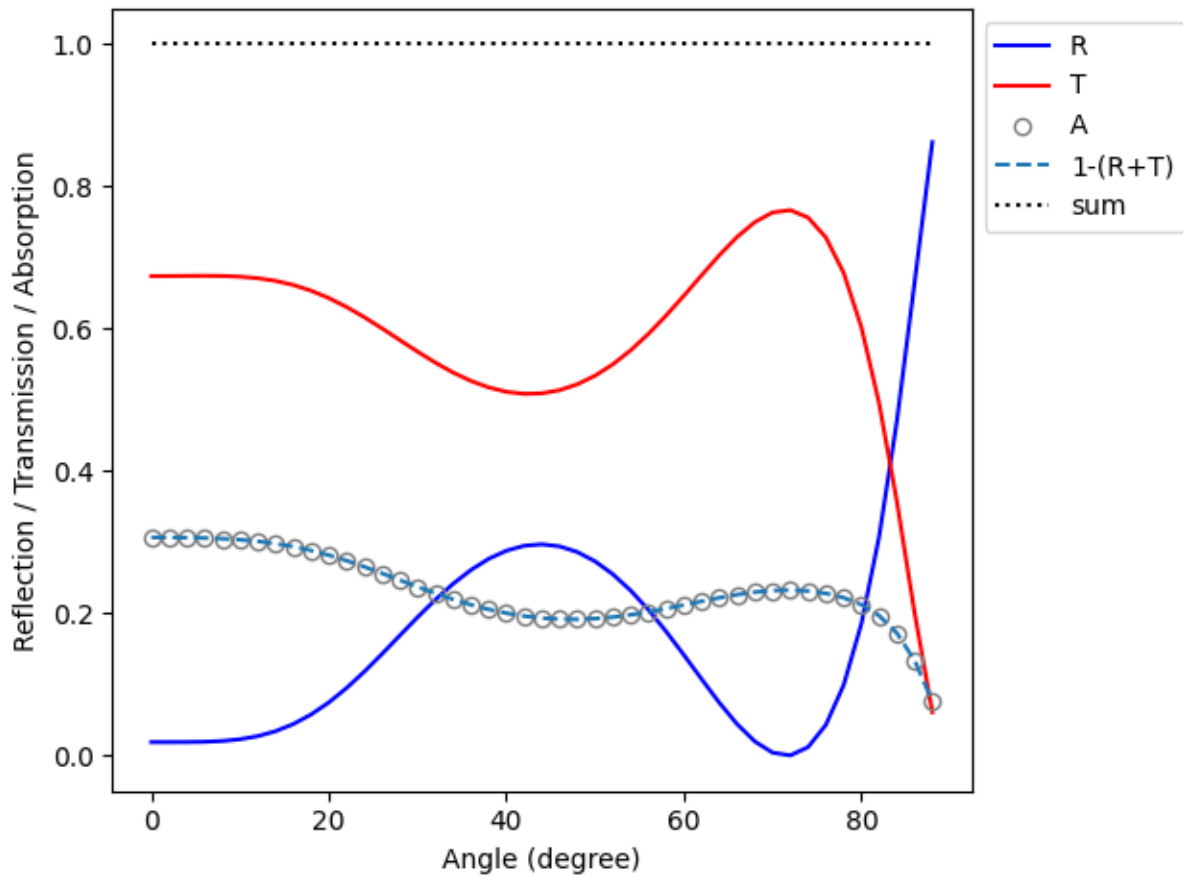
```
Rtm = np.empty(len(theta)) ## reflectivity
Ttm = np.zeros_like(Rtm) ## transmission
Atm = np.zeros_like(Rtm) ## absorption

for ii, thi in enumerate(theta):
    S.SetFrequency(f)
    S.SetExcitationPlanewave(
        IncidenceAngles=(thi, 0), #S4 names are reversed (phi, theta)
        sAmplitude=0.,
        pAmplitude=1.,
        Order=0)
    inc, r = S.GetPowerFlux('AirTop', 0.)
    fw, _ = S.GetPowerFlux('AirBottom', 0.)
    Rtm[ii] = np.abs(-r/inc)
    Ttm[ii] = np.abs(fw/inc)
    fw1, bw1 = S.GetPowerFlux('Slab', 0)
    fw2, bw2 = S.GetPowerFlux('Slab', SlabThick)
    Atm[ii] = np.abs((fw2-fw1-(bw1-bw2))/inc)
```

The reflection and transmission are computed as always at the top and bottom boundary of the simulation. The three most important lines here are the last three:

```
fw1, bw1 = S.GetPowerFlux('Slab', 0)
fw2, bw2 = S.GetPowerFlux('Slab', SlabThick)
Atm[ii] = np.abs((fw2-fw1-(bw1-bw2))/inc)
```

where we store in `fw1`, `bw1` the forward and backward power fluxed *at the top boundary of the layer*, and in `fw2`, `bw2` the forward and backward power fluxed *at the bottom boundary of the layer* (note the `zOffset` parameter set at the value of `SlabThick`). Hence, the absorption in the layer is simply the difference between the in-flowing and out-flowing power. Thanks to the simple form of this simulation, we can quickly check the results:



where we plot the reflection and transmission. We compare the computed absorption (open symbols) to $1 - R - T$ (dashed line), and also check energy conservation (black dots). Using this, we are able to compute the absorption of light in any layer inside the simulation.

2.2.4 Tutorial 4: Dispersive Metal-Insulator-Metal grating

As a last tutorial, we focus on the optical properties of 1D rectangular metallic gratings under TM excitation. The example file can be found under `examples/MIM_DispersiveGrating.py`. This example demonstrates the first patterning method, and shows some of the more complex S^4 options. In addition, it shows some of the utilities functions from S4Utils to make the python API more user friendly.

To install the S4Utils package, refer to the *Download and installation* section.

We import the necessary packages:

```
import time ## to time script execution
import S4 as S4
import numpy as np
import matplotlib.pyplot as plt
```

and two utility packages:

```
import S4Utils.S4Utils as S4Utils
import S4Utils.MaterialFunctions as mat
```

The first one contains various functions to facilitate the use of the S^4 python API, while the second contains python defined permittivity functions. We will run the simulation over actual physical parameters, hence keeping track of the actual values of the frequency, lengths etc. Using micrometers as a base unit in the simulation we define the frequency range in the 12-50 THz range:

```
fmin = 12.0*1e12 ## 400cm-1 en Hz
fmax = 50.0*1e12
f = np.linspace(fmin, fmax, 200)
f0 = f/c_const*1e-6
```

We then define the unit cell using the period of the grating:

```
px = 3.6
ff = 0.8
s = ff*px
NBasis = 41
S = S4.New(Lattice = px,
          NumBasis = NBasis) ### NumBasis <=> halfnpw in RCWA
```

Here `px` is the simulation period, `ff` is the filling factor of the grating, and hence `s` is the size of the metal stripe.

Warning: As noted in various references, metallic gratings and more generally, high index contrast patterns, are generally difficult to model in RCWA. Hence, the number of Fourier coefficients to be considered has to be large to ensure a good convergence. Generally speaking, care must always be taken when setting the `NumBasis` parameter.

The reflectivity spectrum can be calculated for a single angle of incidence, or over a range of angles to get the dispersion relation of the grating modes:

```
theta = np.arange(0,90,5) ### for a dispersion plot
# theta = [0] ## for a single spectrum
```

We get the material permittivities from the *Material database*:

```
epsAu = mat.epsAu(f)
epsGaAs = mat.epsGaAs(f)
```

Note that contrary to the examples above, now we define *dispersive* materials, with a frequency-dependent permittivity. Hence, we store the value of the *complex* permittivity at each frequency of the simulation in an array. The permittivity of a material can also be defined as a tensor, e.g. with diagonal elements $\varepsilon_x, \varepsilon_y = \varepsilon_x, \varepsilon_z$.

Note: The shape of the tensor can either be $3 \times 3 \times \text{len}(f)$ or $\text{len}(f) \times 3 \times 3$

As we wish to use dispersive materials, we will need to update their permittivity each time the frequency of the simulation is changed. A utility function is available in S4Utils to take care of this for all dispersive materials in the simulation: `S4Utils.S4Utils.UpdateMaterials()`, which we will use later in the script.

The materials are set as usual, and we initialize the dispersive materials with the first value of the permittivity array:

```
DisplayThick = 0.5 ## Incident medium thickness
ARThick = 1.0 ## Active region thickness
AuThick = 0.1 ## Gold thickness

### Materials
S.SetMaterial(Name='Air', Epsilon=(1.0 + 0.0*1.0j))
S.SetMaterial(Name='Au', Epsilon=(epsAu[0]))
S.SetMaterial(Name='AR', Epsilon=(epsGaAs[0]))
```

In order to use the function that updates the dispersive materials permittivities, we need to store the material names and permittivity tensors in two lists:

```
# material list for the update function
Mat_list = ['Au', 'AR']
Eps_list = [epsAu, epsGaAs]
```

We then add the layers to the simulation:

```
### Layers
S.AddLayer(Name='top', Thickness = DisplayThick, Material = 'Air') ## incident medium,
↪ air
S.AddLayer(Name='TopGrating', Thickness = AuThick, Material = 'Air') ## grating layer,
↪ will be patterned
S.AddLayer(Name='AR', Thickness = ARThick, Material = 'AR') ## active region layer
S.AddLayer(Name='Bulk', Thickness = 2*AuThick, Material = 'Au') ## bottom mirror
```

Now we need to specify the patterning of the TopGrating layer (see the `S.SetRegionRectangle` function):

```
### Geometry
S.SetRegionRectangle(
    Layer='TopGrating',
    Material='Au',
    Center=(0.0,0.0),
    Angle=0.0,
    Halfwidths=(s/2.,0)) # 1D
```

The patterning method takes the layer to be patterned as an argument, and the material in which the pattern should be made. Note also that since this is a 2D simulation, the halfwidth of the pattern is $s/2$ along the patterning direction, and 0 in the transverse direction.

Before running the simulation, a few additional options have to be set:

```

### Simulation options
S.SetOptions(
    Verbosity=0, ## verbosity of the C++ code
    DiscretizedEpsilon=True, ## Necessary for high contrast
    DiscretizationResolution=8, ## at least 8 if near field calculations
    LanczosSmoothing = True, ## Mabe ?? especially for metals, for near fields
    SubpixelSmoothing=True, ## definitely
    PolarizationDecomposition=True, ## Along with 'normal', should help convergence
    PolarizationBasis='Normal')

```

Description of these options are available in the [RCWA formulations](#) and [S.SetOptions](#) documentations. A priori, evaluation of the Fourier coefficients of the permittivities for anisotropic materials requires discretization of the permittivity patterns, hence we always set the `DiscretizedEpsilon=True` flag to compare simulations run with the same RCWA formulation. The other options improve the convergence with high refractive index contrast, and the spatila resolution of fields computed in each layers.

The simulation now consists in two nested loops:

```

R = np.empty((len(theta),len(f)))

ProgAngle = 0 ## progress in angle sweep
for ii, thi in enumerate(theta): ## angle sweep
    currAngle = int((ii*10)/len(theta)) ## current angle by 10% steps
    if currAngle>ProgAngle:
        print(currAngle) # print progress every 10%
        ProgAngle = currAngle
    ProgF = 0 ## progress in frequency sweep
    for jj, fj in enumerate(f0): ## frequency sweep
        currF = int((jj*10)/len(f0)) ## current frequency by 10% steps
        if currF>ProgF:
            print('\t %d'%currF) # print progress every 10%
            ProgF = currF

        S.SetFrequency(fj) # set the current frequency
        S4Utils.UpdateMaterials(S, Mat_list, Eps_list, fj, f0) # set epsilons
        S.SetExcitationPlanewave(
            IncidenceAngles=(thi,0.),
            sAmplitude=0.,
            pAmplitude=1., ## p-pol plane wave
            Order=0)
        inc, r = S.GetPowerFlux('top', 0.)
        R[ii,jj] = np.abs(-r/inc) ## reflectivity

```

At each loop step, it is crucial to ensure that three main operations are performed: Setting the simulation frequency:

```

S.SetFrequency(fj) # set the current frequency

```

Updating the dispersive materials permittivities using the previously defined `Mat_list` and `Eps_list` lists defined previously:

```

S4Utils.UpdateMaterials(S, Mat_list, Eps_list, fj, f0) # set epsilons

```

and finally ensuring that the excitation is performed with the correct angle and polarization:

```

S.SetExcitationPlanewave(
    IncidenceAngles=(thi,0.),
    sAmplitude=0.,

```

(continues on next page)

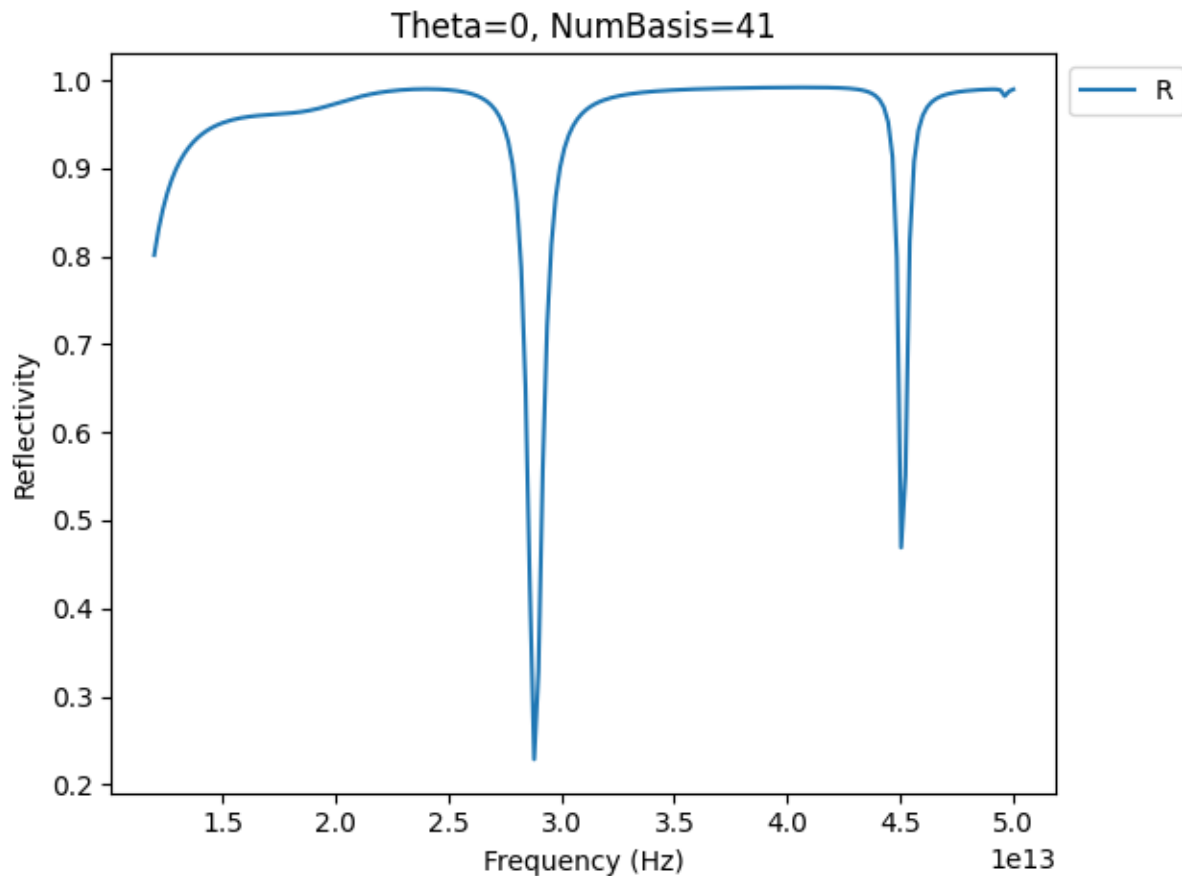
(continued from previous page)

```
pAmplitude=1.,  ## p-pol plane wave
Order=0)
```

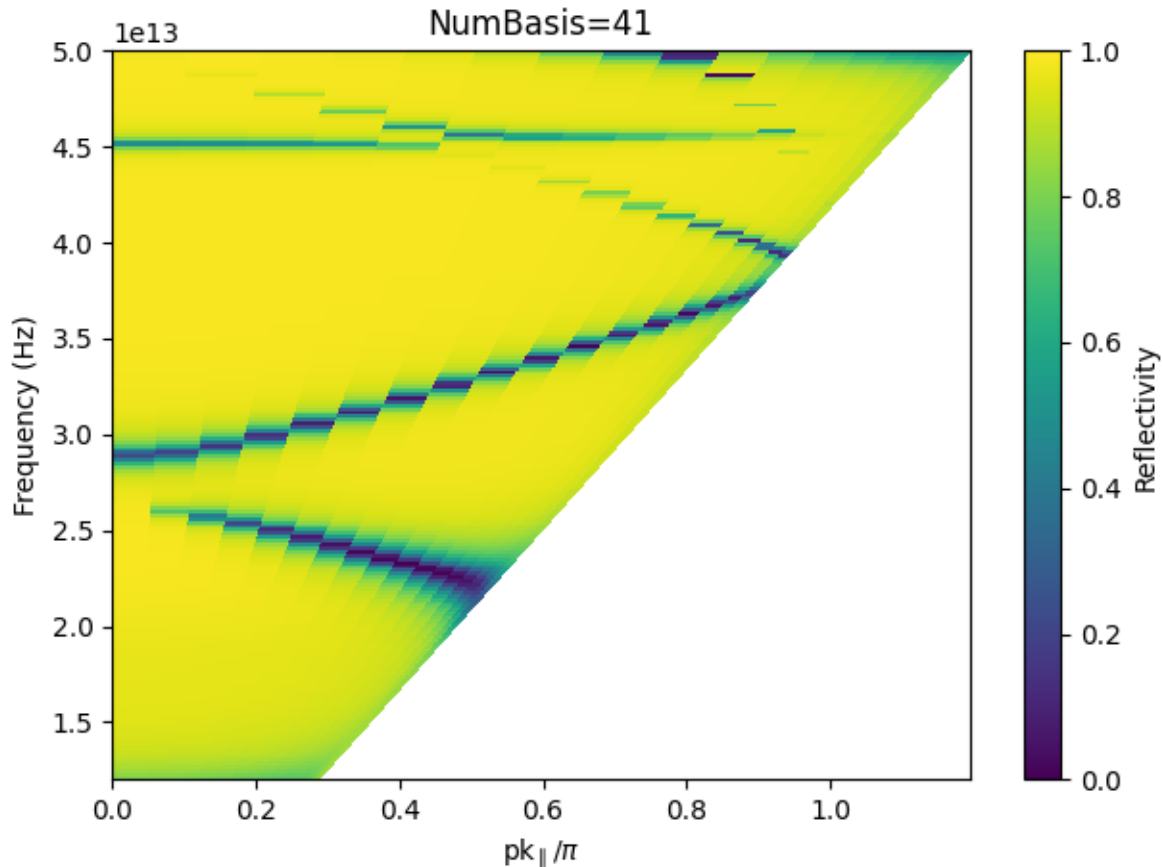
Finally, we plot the results, either for a single spectrum or for a dispersion relation:

```
if len(theta)==1:
    figsp = plt.figure()
    ax = figsp.add_subplot(111)
    ax.plot(f, R.T)
    ax.set_xlabel('Frequency (Hz)')
    ax.set_ylabel('Reflectivity')
else:
    figdisp = plt.figure()
    axm = figdisp.add_subplot(111)
    thm, fm = np.meshgrid(theta,f)
    km = (px*1e-6)*(2*np.pi*fm)/c_const*np.sin(np.deg2rad(thm))/np.pi
    cax = axm.pcolormesh(km, fm, R.T,
                        vmin=0, vmax=1,
                        shading='None')
    axm.set_xlabel('pk$_{\parallel}$/$\pi$')
    axm.set_ylabel('Frequency (cm$^{-1}$)')
    figdisp.tight_layout()
plt.show()
```

which leads e.g. for a normal incidence:



or a dispersion with a 5° step which runs in around one minute on a laptop:



Having computed the spectrum, we might want to look at the field distribution in the structure. The python API directly provides some functions to compute the electric field from a given simulation, especially `S.GetFields` and `S.GetFieldsOnGrid`. The first one computes the electric and magnetic field tensors at a *single* given point in space, while the second computes the electric and magnetic fields tensors on an $x - y$ coordinate. **This function thus only returns correct values in 3D simulations.** To make the computations and visualizations easier, S4Utils provides a set of functions to extract electric and magnetic field profiles along slices (*Simulation Functions*) and plot them in a practical visualization environment (*S4Utils Plotting Utilities*). Additionally, it also provides a way to extract the reconstructed permittivity profile using `S.GetEpsilon`, to help extract an image of the geometry, which can sometimes be hard to picture. This also provides a sense of the spatial resolution of the permittivity decomposition.

Say we want to visualize the profile of the vertical component of the electric field at the 28.8 THz resonance upon normal incidence excitation. We set up the simulation:

```
fplot = 28.8e12 ## frequency at which we plot (SI)
f0plot = fplot/c_const*1e-6 # reduced units
S.SetFrequency(f0plot)
S4Utils.UpdateMaterials(S, Mat_list, Eps_list, f0plot, f0) # update materials
S.SetExcitationPlanewave(
    IncidenceAngles=(0.,0.), ## normal incidence
    sAmplitude=0.,
    pAmplitude=1., ## p-pol plane wave
    Order=0)
```

and define the grid coordinates on which we want to extract the field:

```
resx = 150 ## x-resolution
resz = 150 ## z-resolution
x = np.linspace(-px/2, px/2, resx) ## x coordinates
z = np.linspace(0, TotalThick, resz) ## z coordinates
```

Here we choose to plot the field on a single period, however since S^4 takes care of the periodicity by itself, we could have set arbitrary values for the x-coordinates and plot over several periods. Since this is a 2D simulation, we have to use `S.GetFields` and loop over each coordinate points. This is well take care of by `S4Utils.S4Utils.GetSlice()`:

```
E, H = S4Utils.GetSlice(S, ax1=x, ax2=z, plane='xz', mode='Field')
FigEz = S4Utils.SlicePlot(x, z ,np.real(E[:, :, 2]), hcoord=s/4)
FigEz.ax_2D.set_ylabel('z')
FigEz.ax_2D.set_xlabel('x')
FigEz.axcbar.set_ylabel('Ez')
```

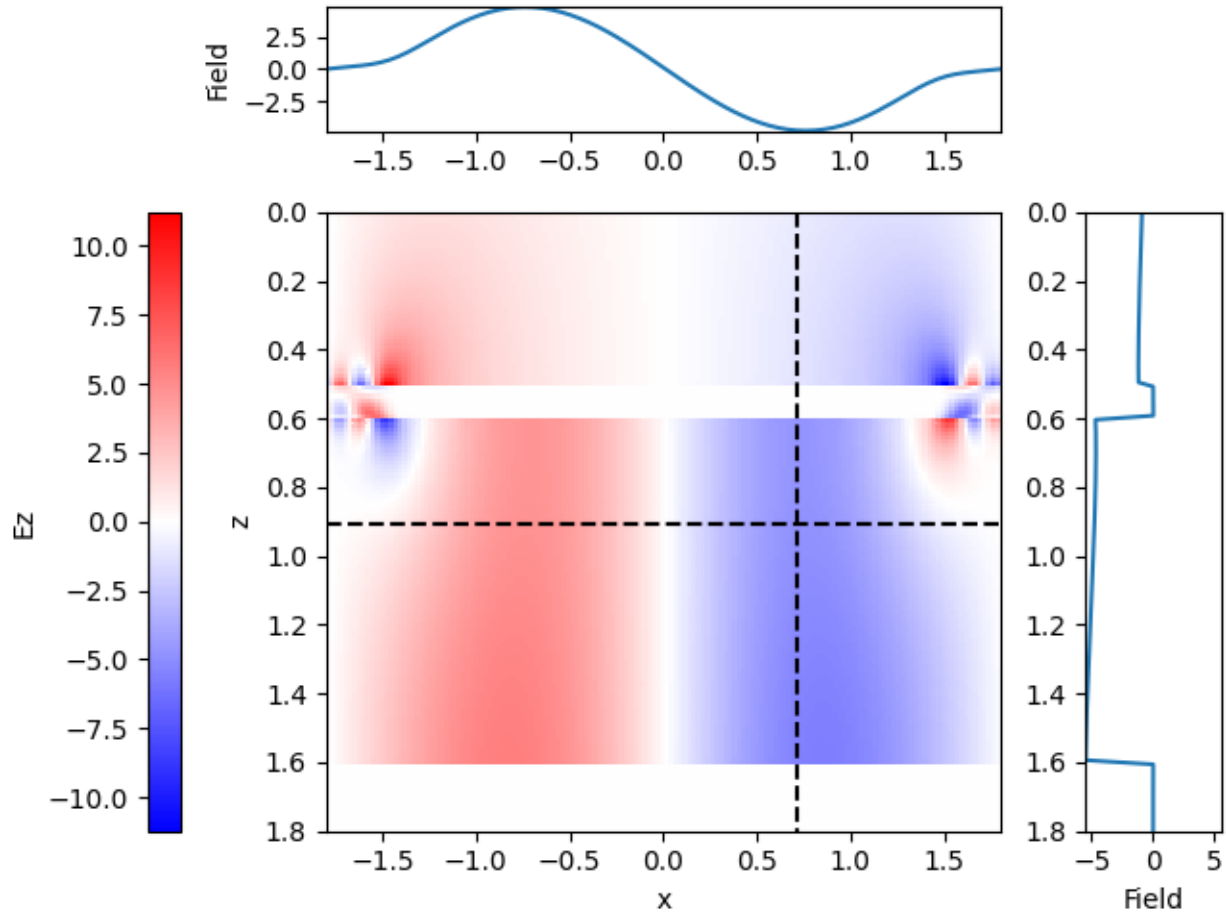
Here the calculation is performed by `S4Utils.GetSlice(S, ax1=x, ax2=z, plane='xz', mode='Field')` where we pass the simulation object `S` as an argument, along with the vector of coordinates `x` and `z`. The `plane` keyword (one of `xy`, `yz`, `xz`) indicates which coordinates to loop over in the C++ function, and the `mode` keyword allows to compute only the fields (`Field`), the permittivity (`Epsilon`) or both (`All`). `E` and `H` are `len(x)*len(y)*3` arrays containing the complex electric and magnetic field amplitude. Hence, the vertical component of the electric field is simply `np.real(E[:, :, 2])`. It could be plotted using a simple `plt.pcolormesh` or `plt.imshow`, however `S4Utils` provides a simple visualization tool that allows to plot fields and slices using `S4Utils.S4Utils.SlicePlot()`:

```
FigEz = S4Utils.SlicePlot(x, z ,np.real(E[:, :, 2]), hcoord=s/4)
```

which is a simple class wrapping around a matplotlib figure instance, and hence can be customized after creation such as changing labels:

```
FigEz.ax_2D.set_ylabel('z')
FigEz.ax_2D.set_xlabel('x')
FigEz.axcbar.set_ylabel('Ez')
```

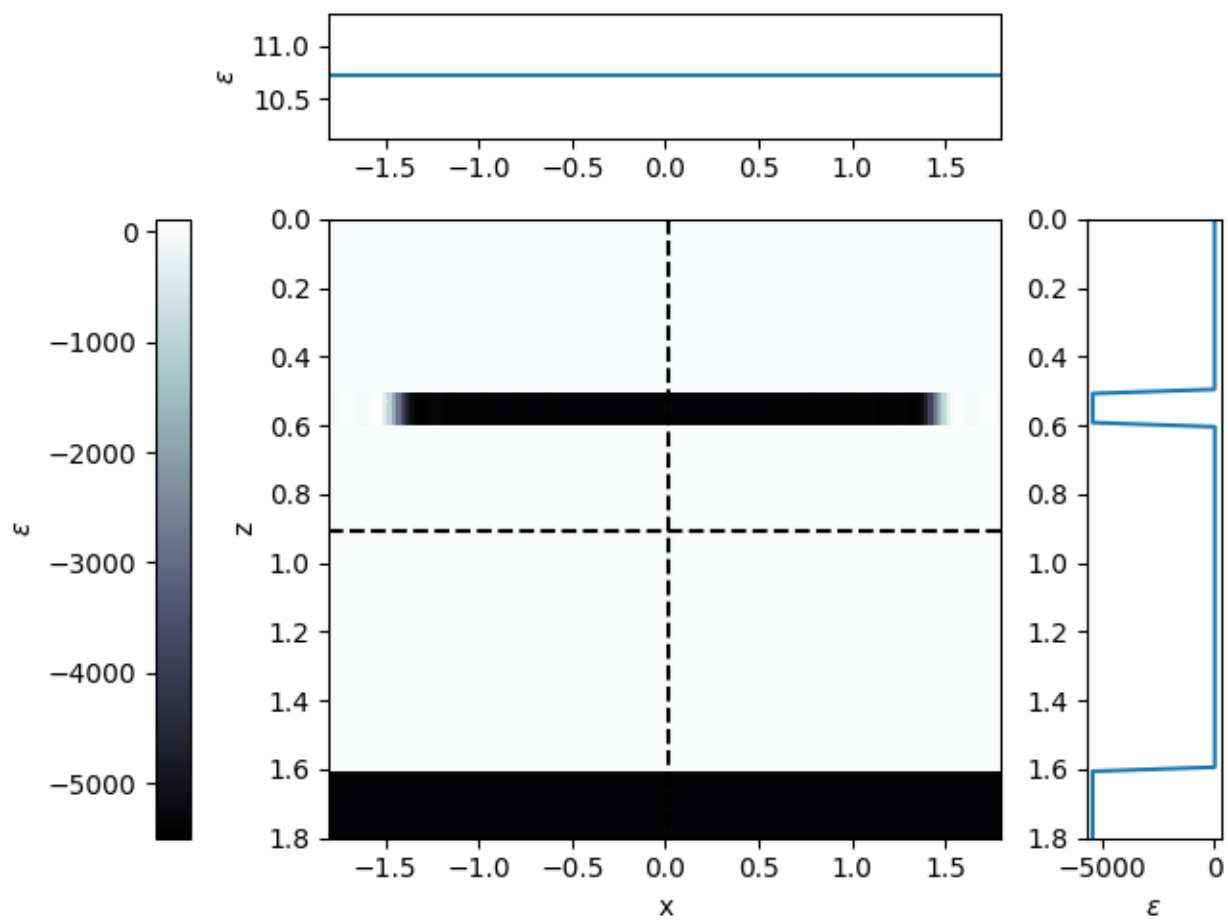
Which allows us to get the 2D electric field plot below, with a symmetrized blue-white-red colormap.



We do the same for the permittivity profile, showing also some more customization options for the `SlicePlot` class:

```
Eps_plot = S4Utils.GetSlice(S, ax1=x, ax2=z, plane='xz', mode='Epsilon')
FigEps = S4Utils.SlicePlot(x, z, np.real(Eps_plot), cmap=plt.cm.bone,
                           sym=False)
FigEps.ax_2D.set_ylabel('z')
FigEps.ax_2D.set_xlabel('x')
FigEps.axcbar.set_ylabel(r'$\varepsilon$')
```

and plot the permittivity profile:



API REFERENCE

3.1 Material database

Created on Sat Mar 28 17:50:31 2020

@author: Mathieu, Paul

Various material definitions are available

`S4Utils.MaterialFunctions.epsAu(f)`

Au permittivity assuming a Drude metal

Parameters `f` (1D array) – frequency (Hz)

Returns `eps` – complex permittivity (len(f))

Return type 1D array

`S4Utils.MaterialFunctions.epsGaAs(f)`

GaAs permittivity assuming a Lorentz model with 2 phonons :param f: frequency (Hz) :type f: 1D array

Returns `eps` – complex permittivity (len(f))

Return type 1D array

`S4Utils.MaterialFunctions.epsAlGaAs(f, xAl)`

AlGaAs permittivity assuming a double 2 phonons model following Adachi, J. Appl. Phys. 58 R1-R29 (1985), table I <https://doi.org/10.1063/1.336070>

Parameters `f` (1D array) – frequency (Hz)

Returns `eps` – complex permittivity (len(f))

Return type 1D array

`S4Utils.MaterialFunctions.epsGaNx(f)`

Gallium nitride (GaN) ordinary axis permittivity (without excitons)

Parameters `f` (1D array) – frequency (Hz)

Returns `eps` – complex permittivity (len(f))

Return type 1D array

`S4Utils.MaterialFunctions.epsGaNz(f)`

Gallium nitride (GaN) extraordinary axis permittivity (without excitons)

Parameters `f` (1D array) – frequency (Hz)

Returns `eps` – complex permittivity (len(f))

Return type 1D array

S4Utils.MaterialFunctions.**epsAlNx**(*f*)

Aluminium nitride (AlN) ordinary axis permittivity (without excitons)

Parameters *f* (1D array) – frequency (Hz)

Returns *eps* – complex permittivity (len(*f*))

Return type 1D array

S4Utils.MaterialFunctions.**epsAlNz**(*f*)

Aluminium nitride (AlN) extraordinary axis permittivity (without excitons)

Parameters *f* (1D array) – frequency (Hz)

Returns *eps* – complex permittivity (len(*f*))

Return type 1D array

S4Utils.MaterialFunctions.**epsZal**(*f*, *eps_w*, *eps_b*, *omega_isb*, *gamma_isb*, *omega_p*, *fw*,
f12=0.96)

Zaluzny model for an active region composed of square QWs (*eps_w*) and barriers (*eps_b*)

Parameters

- *f* (1D array or float) – frequency (Hz)
- *eps_w* (len(*f*) array) – complex permittivity array of the well permittivity
- *eps_barr* (len(*f*) array) – complex permittivity array of the well permittivity
- *omega_isb* (float) – ISB transition pulsation (rad/s) *without plasma shift*
- *gamma_isb* (float) – ISB broadening
- *omega_p* (float) – plasma pulsation (rad/s)
- *fw* (float) – filling fraction (QW thick/period thick)
- *f12* (float) – Oscillator strength. The default is 0.96 (infinite square well)

Returns

- *eps_xx* (len(*f*) array) – complex in-plane permittivity
- *eps_zz* (len(*f*) array) – complex out-of-plane permittivity

S4Utils.MaterialFunctions.**epsPQW**(*f*)

Doped Parabolic Quantum Well permittivity Paul?

Parameters *f* (1D array) –

Returns

Return type None

S4Utils.MaterialFunctions.**epsDrude**(*f*, *fp*, *gammap*, *epsinf*=1.0)

Drude model for a material with free electrons

Parameters

- *f* (1D array) – frequency (Hz)
- *fp* (float) – plasma frequency (Hz)
- *gammap* (float) – damping (collision rate)
- *epsinf* (float) – high-frequency permittivity. Defaults to 1.0 (metal).

Returns *eps* – complex permittivity

Return type len(f) array

S4Utils.MaterialFunctions.**eps2Phonons** (*f*, *wT*, *wL*, *gammaT*, *gammaL*, *eps_inf*)
2 phonons model for a semiconductor material

Parameters

- **f** (*1D array*) – frequency (Hz)
- **wT** (*float*) – Transverse phonon pulsation (rad/s)
- **wL** (*float*) – Longitudinal phonon pulsation (rad/s)
- **gammaT** (*float*) – Transverse phonon damping rate (rad/s)
- **gammaL** (*float*) – Longitudinal phonon damping rate (rad/s)
- **eps_inf** (*float*) – high-frequency permittivity. Defaults to 1.0 (metal).

Returns **eps** – complex permittivity

Return type len(f) array

S4Utils.MaterialFunctions.**epsLorentzPhonon** (*f*, *wT*, *wL*, *gamma*, *eps_inf*)
Lorentz phonon model for a semiconductor material

Parameters

- **f** (*1D array*) – frequency (Hz)
- **wT** (*float*) – Transverse phonon pulsation (rad/s)
- **wL** (*float*) – Longitudinal phonon pulsation (rad/s)
- **gamma** (*float*) – Phonon damping rate (rad/s)
- **eps_inf** (*float*) – high-frequency permittivity. Defaults to 1.0 (metal).

Returns **eps** – complex permittivity

Return type len(f) array

S4Utils.MaterialFunctions.**omegaP_2D** (*N2D*, *meff*, *eps_i*, *w_QW*)
Plasma pulsation (in units of rad/s) for an equivalent 2D doping N2D

Parameters

- **N2D** (*float*) – equivalent sheet carrier density (m⁻²)
- **meff** (*float*) – effective mass coefficient (no units)
- **eps_i** (*float*) – high-frequency permittivity
- **w_QW** (*float*) – quantum well thickness (m)

Returns **wp** – plasma pulsation (rad/s)

Return type float

S4Utils.MaterialFunctions.**omegaP_3D** (*N3D*, *meff*, *eps_i*)
Plasma pulsation (in units of rad/s) for an equivalent 3D doping N3D

Parameters

- **N3D** (*float*) – carrier concentration (m⁻³)
- **meff** (*float*) – effective mass coefficient (no units)
- **eps_i** (*float*) – high-frequency permittivity

Returns `wp` – plasma pulsation (rad/s)

Return type float

3.2 S4Utils functions references

Created on Thu Apr 2 12:05:13 2020

@author: mathieu

3.2.1 Basic Utility functions

`S4Utils.S4Utils.totuple(a)`

Converts lists and array to a tuple form. Usefull since S4 mostly accepts tuples instead of numpy arrays.

Parameters `a` (*array-like, list or float*) – Array-like parameter to cast to a tuple

Returns `ta` – elements of a cast into a tuple of same ‘shape’.

Return type tuple

:raises `TypeError`:: If the type of `a` cannot be cast into a tuple form, returns a

`S4Utils.S4Utils.UpdateMaterials(S, Mat_list, Eps_list, fi, f0)`

Updates a Simulation to set all refractive indices at given frequency

Parameters

- `S` (*S4.Simulation*) –
- `Mat_list` (*list*) – list of string containing the name of the materials to update. These are typically the ones passes in the ‘Material’ argument for the `S4.Simulation.SetMaterial()` function
- `Eps_list` (*list*) – list of arrays containing the permittivities. each element is a 1D array or a 3x3xlen(f) array
- `fi` (*float*) – frequency at which the simulation is set (reduced units)
- `f0` (*1Darray*) – array of reduced frequencies over which the simulation runs

Returns

Return type None.

3.2.2 Simulation Functions

2D Slicing functions

`S4Utils.S4Utils.GetField_xy(S, res, zs)`

Get E and H fields in an xy slice. Built-in S4 function that requires some care and a tweak, but extra efficient !!

Parameters

- `S` (*S4.Simulation*) –
- `res` (*tuple*) – (resx, resy) tuple of number of points along x and y Note that it corresponds to *resy lines* and *resx columns*
- `zs` (*float*) – z-coordinate at which to slice the field

Returns**Return type** None.`S4Utils.S4Utils.GetField_xz (S, x, y, z, ys=0.0)`

Get E and H fields in an xz slice. Wraps around GetField_xy which is hard-coded in the C code and thus much faster. *BUT* you should check that it matches with GetSlice at least once because the C function GetField-sOnGrid tends to be touchy.

Parameters

- **S** (*S4.Simulation Object*) – Simulation to run
- **x** (*1Darray*) – 1Darray of x-coordinates
- **y** (*1Darray*) – 1Darray of y-coordinates
- **z** (*1Darray*) – 1Darray of z-coordinates
- **ys** (*float, optional*) – y-coordinate of the slice. Defaults to 0 (center of the unit cell).

Returns

- **E_slice** (*len(z) by len(x) by 3 array*) – Array of the electric field vector on the slice
- **H_slice** (*len(z) by len(x) by 3 array*) – Array of the magnetic field vector on the slice

`S4Utils.S4Utils.GetField_yz (S, x, y, z, xs=0.0)`

Get E and H fields in an yz slice. Wraps around GetField_xy which is hard-coded in the C code and thus much faster. *BUT* you should check that it matches with GetSlice at least once because the C function GetField-sOnGrid tends to be touchy.

Parameters

- **S** (*S4.Simulation Object*) – Simulation to run
- **x** (*1Darray*) – 1Darray of x-coordinates
- **y** (*1Darray*) – 1Darray of y-coordinates
- **z** (*1Darray*) – 1Darray of z-coordinates
- **xs** (*float, optional*) – x-coordinate of the slice. Defaults to 0 (center of the unit cell).

Returns

- **E_slice** (*len(z) by len(y) by 3 array*) – Array of the electric field vector on the slice
- **H_slice** (*len(z) by len(y) by 3 array*) – Array of the magnetic field vector on the slice

`S4Utils.S4Utils.GetSlice (S, ax1, ax2, axs=0, plane='xz', mode='All')`

Slice field and/or epsilon in an arbitrary slice along 2 axis at a given position on the third. Point-by-point computation version. There is no built-in function for this, so it is much slower. However it is less ambiguous than the GetField methods and can be used to check/debug.

Parameters

- **S** (*S4.Simulation*) –
- **ax1** (*1Darray*) – first axis to define the slice plane
- **ax2** (*1Darray*) – second axis to define the slice plane
- **axs** (*float*) – coordinate along thirs axis on which to slice
- **plane** (*str*) – 'xz', 'yz' or 'xz' to choose which slice it is

- **mode** (*str*) – either ‘Field’, ‘Epsilon’, ‘All’ to choose which quantity to slice

3D Volume functions

`S4Utils.S4Utils.GetFieldVolume (S, x, y, z)`

Get the electric field vector in the whole simulation volume

Parameters

- **S** (*S4.Simulation object*) – Simulation object to run
- **x** (*1Darray*) – 1Darray of x-coordinates
- **y** (*1Darray*) – 1Darray of y-coordinates
- **z** (*1Darray*) – 1Darray of z-coordinates

Returns

- **E_vol** (*len(x) by len(y) by len(z) by 3 array*) – Array of electric field in the simulation volume
- **H_vol** (*len(x) by len(y) by len(z) by 3 array*) – Array of magnetic field in the simulation volume

`S4Utils.S4Utils.GetEpsilonVolume (S, x, y, z)`

Get the reconstructed permittivity in the whole simulation volume

Parameters

- **S** (*S4.Simulation object*) – Simulation object to run
- **x** (*1Darray*) – 1Darray of x-coordinates
- **y** (*1Darray*) – 1Darray of y-coordinates
- **z** (*1Darray*) – 1Darray of z-coordinates

Returns **Eps_vol** – Array of the permittivity in the simulation volume

Return type len(x) by len(y) by len(z) array

3.2.3 S4Utils Plotting Utilities

The SlicePlot class

`class S4Utils.S4Utils.SlicePlot (a1, a2, Field, cmap=matplotlib.pyplot.cm.bwr, hcoord=None, vcoord=None, sym=True)`

Helper to plot a slice with linecuts at arbitrary positions

Parameters

- **a1** (*1Darray*) – 1Darray for the *horizontal* axis of the 2D plot (typically x)
- **a2** (*1Darray*) – 1Darray for the *vertical* axis of the 2D plot (typically y or z)
- **Field** (*2Darray*) – 2Darray to be plotted (len(a2) x len(a1))
- **cmap** (*matplotlib.colors.Colormap object*) – The colormap to pass to *matplotlib.pcolormesh* for plotting. Defaults to *matplotlib.cm.bwr*.
- **hcoord** (*float, optional*) – Horizontal coordinate for the vertical slice. Defaults to the center of the axe coordinate array given (a1).

- **vcoord** (*float, optional*) – Vertical coordinate for the horizontal slice. Defaults to the center of the axe coordinate array given (a2).
- **sym** (*bool, optional*) – Flag to force symmetry in the colormap or leave it automated

a11Darray for the *horizontal* axis of the 2D plot (typically x)**Type** 1Darray**a2**1Darray for the *vertical* axis of the 2D plot (typically y or z)**Type** 1Darray**Field**

2Darray to be plotted (len(a2) x len(a1))

Type 2Darray**hcoord**

Horizontal coordinate for the vertical slice. Defaults to the center of the axe coordinate array given (a1).

Type float, optional**vcoord**

Vertical coordinate for the horizontal slice. Defaults to the center of the axe coordinate array given (a2).

Type float, optional**sym**

Flag to force symmetry in the colormap or leave it automated

Type bool, optional**figslice**

The matplotlib figure in which the plot is drawn

Type matplotlib.figure**fignum**

The number of the figure as retrieved by matplotlib.figure.number

Type int**ax_2D**

The axes in which the 2D slice is drawn

Type matplotlib.axes**cax**

The axes in which the colorbar for ax_2D drawn

Type matplotlib.axes**ax_cbar**

The matplotlib.colorbar instance

Type matplotlib.colorbar**ax_alslice**

The axes in which the slice along a2 at a fixed a1 position is drawn

Type matplotlib.axes**lv**The line of the *vertical* cut, e.g. along a2

Type matplotlib.line

ax_a2slice

The axes in which the slice along a1 at a fixed a2 position is drawn

Type matplotlib.axes

lh

The line of the *horizontal* cut, e.g. along a1

Type matplotlib.line

lslice_h

The line showing the position of the horizontal slice in ax_2D

Type matplotlib.line

lslice_v

The line showing the position of the vertical slice in ax_2D

Type matplotlib.line

GUIButton

The button to hide/show the GUI

Type matplotlib.widgets.CheckButtons

Notes

For a 3D stack of slices one should move to Plotly instead of matplotlib <https://nbviewer.jupyter.org/github/empet/Plotly-plots/blob/master/Plotly-Slice-in-volumetric-data.ipynb>

The GUISlicePlot class

class S4Utils.S4Utils.GUISlicePlot (*parentfig*)

Generates an auxiliary figure to interact with a SlicePlot. The GUI side plot allows to move the vertical and horizontal sliders to show slices at different locations.

Parameters *parentfig* (*S4Utils.S4Utils.SlicePlot* instance) – The SlicePlot figure to which the GUI applies.

parentfig

The SlicePlot figure to which the GUI applies.

Type *S4Utils.S4Utils.SlicePlot* instance

figGUI

The matplotlib.figure in which the GUI is drawn

Type matplotlib.figure instance

ax1

The axes for the first slider linked to parentfig.a1

Type matplotlib.axes

ax2

The axes for the first slider linked to parentfig.a2

Type matplotlib.axes

s1_ax1

The slider linked to the parentfig.a1 axis

Type matplotlib.widgets.Slider

S1_ax2

The slider linked to the parentfig.a2 axis

Type matplotlib.widgets.Slider

Returns

Return type None.

Todo: Add a frequency input textbox to change the frequency and update plot

3D Plotting Function

`S4Utils.S4Utils.Plot3DSlices (axes, DataVol, points, InteractiveSlices=False)`

3D rendering of 3 slices in orthogonal planes.

Parameters

- **axes** – x, y and z 1Darray vectors of coordinates on which the data is calculated. Usually passed as (x,y,z) or `np.array((x,y,z))`
- **DataVol** (`len(x) by len(y) by len(z) numpy array`) – Volumetric data in numpy array to be plotted Usually obtained using `S4Utils.S4Utils.GetFieldVolume()` or `S4Utils.S4Utils.GetEpsilonVolume()` functions.
- **points** (`tuple, list or array`) – point at which to slice the volume data. Usually (xs, ys, zs), in real space coordinates
- **InteractiveSlices** (`bool (opt)`) – Boolean flag to activate PyVista's defaults 3D slicing. Defaults to False.

Returns

Return type None.

Notes

Follows <https://docs.pyvista.org/examples/00-load/create-uniform-grid.html> and https://docs.pyvista.org/plotting/widgets.html#pyvista.WidgetHelper.add_mesh_slice_orthogonal

3.3 epsInterpolator functions references

Created on Mon Mar 30 09:12:22 2020

@author: mathieu

`S4Utils.epsInterpolator.eps_interpolator (f, fp, x1, x2, nk=False)`

Returns the complex permittivity interpolated from tabulated data, either in (n,k) files or in (eps1,eps2) files.

Parameters

- **f** (`1D array`) – 1D array of frequency at which to interpolate the data
- **fp** (`1D array`) – Array of frequency for the tabulated data

- **x1** (*1D array*) – Array of tabulated data (n or eps1)
- **x2** (*1D array*) – Array of tabulated data (k or eps2)
- **nk** (*Bool, optional*) – Flag. True if the data is in (n,k) form, False if in (eps1, eps2) form. Defaults to False

Returns **eps** – 1D array of (complex) permittivity (len(f))

Return type 1D array

`S4Utils.epsInterpolator.eps_Au(f)`

Ordal tabulated values for gold

Parameters **f** (*1D array*) – Frequency

Returns **eps** – Complex permittivity of Au (Ordal tabulated data)

Return type 1D array

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

S

`S4Utils.epsInterpolator`, [31](#)
`S4Utils.MaterialFunctions`, [23](#)
`S4Utils.S4Utils`, [26](#)

A

a1 (*S4Utils.S4Utils.SlicePlot* attribute), 29
a2 (*S4Utils.S4Utils.SlicePlot* attribute), 29
ax1 (*S4Utils.S4Utils.GUISlicePlot* attribute), 30
ax2 (*S4Utils.S4Utils.GUISlicePlot* attribute), 30
ax_2D (*S4Utils.S4Utils.SlicePlot* attribute), 29
ax_al slice (*S4Utils.S4Utils.SlicePlot* attribute), 29
ax_a2 slice (*S4Utils.S4Utils.SlicePlot* attribute), 30
ax_cbar (*S4Utils.S4Utils.SlicePlot* attribute), 29

C

cax (*S4Utils.S4Utils.SlicePlot* attribute), 29

E

eps2Phonons () (in module *S4Utils.MaterialFunctions*), 25
eps_Au () (in module *S4Utils.epsInterpolator*), 32
eps_interpolator () (in module *S4Utils.epsInterpolator*), 31
epsAlGaAs () (in module *S4Utils.MaterialFunctions*), 23
epsAlNx () (in module *S4Utils.MaterialFunctions*), 24
epsAlNz () (in module *S4Utils.MaterialFunctions*), 24
epsAu () (in module *S4Utils.MaterialFunctions*), 23
epsDrude () (in module *S4Utils.MaterialFunctions*), 24
epsGaAs () (in module *S4Utils.MaterialFunctions*), 23
epsGaNx () (in module *S4Utils.MaterialFunctions*), 23
epsGaNz () (in module *S4Utils.MaterialFunctions*), 23
epsLorentzPhonon () (in module *S4Utils.MaterialFunctions*), 25
epsPQW () (in module *S4Utils.MaterialFunctions*), 24
epsZal () (in module *S4Utils.MaterialFunctions*), 24

F

Field (*S4Utils.S4Utils.SlicePlot* attribute), 29
figGUI (*S4Utils.S4Utils.GUISlicePlot* attribute), 30
fignum (*S4Utils.S4Utils.SlicePlot* attribute), 29
figslice (*S4Utils.S4Utils.SlicePlot* attribute), 29

G

GetEpsilonVolume () (in module *S4Utils.S4Utils*),

28

GetField_xy () (in module *S4Utils.S4Utils*), 26
GetField_xz () (in module *S4Utils.S4Utils*), 27
GetField_yz () (in module *S4Utils.S4Utils*), 27
GetFieldVolume () (in module *S4Utils.S4Utils*), 28
GetSlice () (in module *S4Utils.S4Utils*), 27
GUIButton (*S4Utils.S4Utils.SlicePlot* attribute), 30
GUISlicePlot (class in *S4Utils.S4Utils*), 30

H

hcoord (*S4Utils.S4Utils.SlicePlot* attribute), 29

L

lh (*S4Utils.S4Utils.SlicePlot* attribute), 30
lslice_h (*S4Utils.S4Utils.SlicePlot* attribute), 30
lslice_v (*S4Utils.S4Utils.SlicePlot* attribute), 30
lv (*S4Utils.S4Utils.SlicePlot* attribute), 29

M

module
S4Utils.epsInterpolator, 31
S4Utils.MaterialFunctions, 23
S4Utils.S4Utils, 26

O

omegaP_2D () (in module *S4Utils.MaterialFunctions*), 25
omegaP_3D () (in module *S4Utils.MaterialFunctions*), 25

P

parentfig (*S4Utils.S4Utils.GUISlicePlot* attribute), 30
Plot3DSlices () (in module *S4Utils.S4Utils*), 31

S

S4Utils.epsInterpolator
module, 31
S4Utils.MaterialFunctions
module, 23
S4Utils.S4Utils
module, 26

`sl_ax1` (*S4Utils.S4Utils.GUISlicePlot attribute*), 30

`sl_ax2` (*S4Utils.S4Utils.GUISlicePlot attribute*), 31

`SlicePlot` (*class in S4Utils.S4Utils*), 28

`sym` (*S4Utils.S4Utils.SlicePlot attribute*), 29

T

`totuple()` (*in module S4Utils.S4Utils*), 26

U

`UpdateMaterials()` (*in module S4Utils.S4Utils*), 26

V

`vcoord` (*S4Utils.S4Utils.SlicePlot attribute*), 29